Milestone 3 Prototype build

I will first construct the <theory-of-the-program>, then generate <program text> only after the theory is explicit.

OPERATIONAL_DIAGNOSTIC

Substantives_Demolished Replace vague uses of functionality, portability and understanding with the actions and records below.

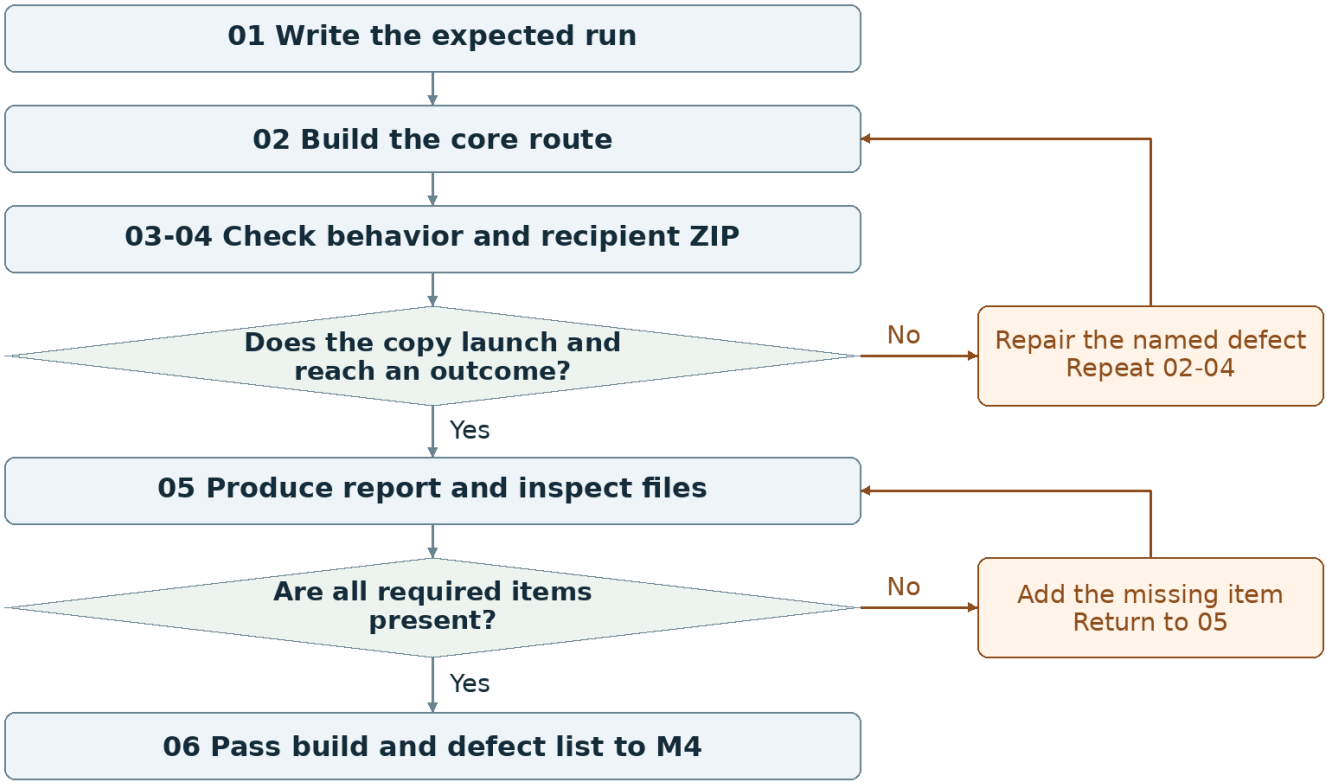
Occult_States_Removed Replace it works and we understand the code with a recorded launch, input sequence, observed result and partner trace through named files.

The_Language_Game Predict a run, execute that run in the packaged copy, and repair any difference before recording a pass.

CONTROL_SURFACE_DRAFT

Build a complete minimal interaction and test the actual recipient package on another computer.

Start with Open the M2 brief, transition table, program state diagram, mockups, asset plan and unresolved technical checks. Keep the required Processing p5.js mode project intact.



Use the numbered procedure; repeat checks only where the repair affects their result.

Required package Submit one working-project ZIP plus one group PDF of at least two pages.

Assign maker _____ and checker _____. Swap roles at the next task. Leave checks pending until performed.

Milestone 3 Write the rules

1 <Initial Interpretation>

Write the expected sequence: <player> [activates] <Start>; <input handler> [sets] <main state>; <player> [uses] <control>; <named handler> [updates] <data>; <outcome handler> [checks] <ending test> and [records] <outcome>. Use actual handler names when they exist. List optional features that may remain placeholders.

2 <Theory Skeleton>

Record	What to write
<Entities>	List actual runtime data, starting values and the files that store them. Mark planned data separately.
[Operations]	Identify the handler that accepts input, validates it, changes data and causes displayed feedback.
<States>	Implement a title, main interaction and at least one fail or success state; retain at least three screens/states overall.
<Conditions>	Write the exact input guards and ending tests. Specify behavior after an outcome and for replay if provided.
<Invariants>	Carry forward scoped rules from M2. Check valid value ranges after accepted and rejected inputs. Record expected reset values as the postconditions of replay.
<Constraints>	Keep controls visible. Record actual authoring versions and recipient launch route. Require no extra software downloads and no complex recipient setup; preserve cross-OS operation.

Use conditions to decide whether an action or transition is allowed. State the scope of each invariant and how to check it. Keep submission requirements separate from runtime rules.

3 <Assumption Ledger>

Status	Action
<safe>	Enter an unverified premise only with its basis and limited consequence if wrong. Keep agreed rules in the plan and observed results in check records.
<uncertain>	Mark timing, rapid input, dependencies and every untested OS/browser combination not tested.
<requires-user-decision>	Resolve any repair that changes the player's rule in M2 before implementing it.

Record {Processing version, mode version, p5.js version, dependency, intended launch method, test environment}. Do not infer recipient operation from the editor's Run button.

Record the next open item Claim _____ Status _____
Owner _____ Evidence/check _____ Next action _____

Write <entity> for a person, object, value or state; [operation] for an action; and {record} for related fields. Record planned, observed, failed or not tested beside each claim. Do not mark a check passed until a person performs it and records the result.

Milestone 3 Run the procedure

4 <Operational Description>

Read each action aloud before doing it. The maker performs the action; the checker records the named result. Use the return instruction when a check fails.

Step	Perform and save	Check and return
01 Write the expected run	The maker writes starting values, inputs, next states, visible output and ending before running code. The checker compares this with M2.	If predictions conflict, revise the rule before building it.
02 Build the core route	Implement title, visible controls, main action, response and a reachable fail or success state in Processing p5.js mode. Use placeholders that still communicate the action and result.	Keep at least three screens/states. Pause styling work when it delays a missing transition.
03 Run behavior checks	Execute the predicted route in the authoring environment. Try a blocked input, rapid repeated input and replay if supported. Write expected and observed outputs separately.	Repair the smallest failing operation; replay its input sequence. Record a designed player loss separately from a software defect.
04 Test distribution	Create the actual ZIP. Extract it in a fresh location on another computer; follow only the included launch instructions. Record OS/browser, network condition, dependencies, inputs and result.	Repair missing files or launch problems, recreate the ZIP, extract again and retest. Leave untested environments marked not tested.
05 Produce the report	Write what works and what is missing. Add current screenshots, launch/test record, short progress note, credits and AI statement. Export at least two pages and reopen both deliverables.	Check the screenshots and report identify the same build inside the ZIP.
06 Hand off	Pass the exact prototype, rule changes, distribution record and ordered missing-work list to M4. Each partner traces one input through the submitted code.	Resolve a missing required route before marking the prototype ready for the alpha stage.

5 <Failure Description>

Observed problem	Required response
Code runs only in the original folder	Repair dependencies and relative paths in step 04; test a newly extracted ZIP.
Rules change during drawing or repeat unexpectedly	Locate the data-changing handler in step 03; define when an action counts and test that rule.
Simple launch needs extra setup	Simplify the dependency or resolve the launch method. Do not replace the required working ZIP with a video or an untested hosted link.

Use the map A no branch returns to the named task. Repair the recorded problem and repeat that check; do not mark the stage passed because the planned time window ended.

Milestone 3 Test changes and produce the work

6 <Change Test>

Ask: "What would change if the requirements changed?" The maker writes the expected result first. The checker performs or traces the change and records the result. Swap roles for the second scenario.

Modification	Run this check
Reduce frame rate	Record whether the timing code uses elapsed time or frame count. Run at a lower frame rate in a test copy. Compare duration and action counts with the written rule; repair unintended timing changes and restore normal settings.
Repeat input rapidly	Write which presses count in the current state and which are ignored. Send the repeated input. Compare data changes and transitions with that count; record and repair duplicate triggers.

Save {version or design record, starting condition, input/change, expected result, observed result, pass/fail/not tested, next action}. Use paper traces in M1-M2; report executed checks only when they occurred.

7 <Implementation Plan>

Give one partner a small behavior change and the other its prediction/check. Swap at the next operation. Keep initialization, input, rule updates and drawing responsibilities identifiable. Bundle compatible dependencies and test the chosen recipient route; record network requirements rather than assuming offline support.

8 <Program Text>

Write only the next small operation after its inputs, conditions and expected output are recorded. For AI-assisted examples or repairs, study, adapt and rewrite where appropriate; explain the change to the partner and add assistance comments. Do not replace the whole sketch with generated code.

9 <Theory-Code Mapping>

Element	Record the connection
<types>	Write actual variable/data names next to their domain meaning.
<functions>	Locate initialization, input, rule update, drawing and ending checks.
<classes>	Record used class responsibilities or not used; classes are optional.
<tests>	Link input sequences to expected and observed outputs and build ID.
<comments>	Explain non-obvious rules, dependencies and AI-assisted changes.
<configuration>	Identify tunable values and checks to repeat after changing them.

Use actual file and function names when code exists. Label planned counterparts proposed; write not applicable when a structure is absent.

Carry forward the course rules

Keep the project connected to WarGames. Build in Processing using p5.js mode; accept interactive input; include at least three distinct screens or states; provide a start and reachable outcome; credit external material; organize and comment the code.

Record actual AI use at every milestone, including use of this manual if adopted. Use AI for bounded brainstorming, explanation, small examples you study and adapt, specific debugging or review. Author the project as a group; do not request an entire generated project. Identify assisted code and your changes in comments.

Milestone 3 Submit and hand off

10 <Residual Human Theory>

Each partner selects one input, predicts its effect, finds the responsible function and data, and compares the prediction with a run. Keep the design reason and any untested behavior in the handoff note. Do not convert a successful test on one computer into a claim about every operating system.

Check the required submission

- One ZIP containing the working prototype and files needed for its documented launch.
- A title screen, main interaction loop and at least one fail OR success state; at least three screens/states overall.
- One group PDF of at least two pages with what works and what is missing.
- At least one actual prototype screenshot, a short progress note, references and AI Use Statement.
- Provide cross-OS operation on another computer without extra software downloads or complex setup. Test the extracted ZIP; record actual OS/browser, network needs and result. Label untested environments not tested.

Suggested report order: page 1 current behavior and screenshots; page 2 or later missing work, launch/check evidence, progress note and AI statement. Keep the report separate from the required ZIP.

Completion check The checker launches the recipient copy by the written simple method, starts from the title, uses the shown controls and reaches the intended outcome. The required ZIP, PDF and accurate records all exist.

Progress note Name the current version; state what was completed and checked, what is missing, and the next action with its owner. Add the screenshot/capture supporting the current status.

Complete the AI Use Statement

Field	Enter the actual record
Tool and dates	Name the tool, or record no AI used only if accurate. Enter actual dates of use.
Request and affected work	Summarize the request. Name the text, image, planning record or code file affected.
Kept changed rejected	Record the output retained, edits or rewrites made, and material discarded.
Group confirmation	Recommended check: each partner traces actual submitted code; for planning-only work, trace the proposed interaction. Identify image sources, retained edits and displayed information. Then complete the required group confirmation covering code and images; leave it pending until both partners can confirm it.

If this AI-produced manual is adopted, record ChatGPT, October 21 2026, milestone planning and operational rewrite, and your actual kept/changed/rejected choices. Do not copy a completed student confirmation from this manual.

Handoff Carry the tested package, implementation-to-rule map, observed defects and missing features to M4. Preserve untested status where evidence is absent.

Record: maker _____ checker _____ version _____
Checks still pending _____ submitted file(s) _____ upload status _____

Requirements source: supplied WarGames project brief and milestone instructions. Treat roles, file names, internal checks and relative-week allocations as recommended working procedures. Use corrected deadlines from the course; none are asserted here.